Unidad Profesional Interdisciplinaria de Ingeniería y Ciencias Sociales y Administrativas
IPN

Materia

Abstracción y uso de datos

Integrantes

Adame Barranco Fernanda Harumi

Gallegos Montiel Isaac

Gonzalez Flores Isaac Yael

Índice

Ejercicio 16: Ordenamiento Burbuja 3
Ejercicio 17: Ordenamiento Merge Sort 3
Ejercicio 18: Ordenamiento Quick Sort 3
Ejercicio 19: Ordenamiento Indirecto por Burbuja 4
Ejercicio 20: Ordenamiento Indirecto por Merge Sort 4
Ejercicio 21: Ordenamiento indirecto por quick sort 4
Ejercicio 1: Punteros para el manejo indirecto con clases y estructuras 5
Ejercicio 2: ADT (Abstract Data Type - Tipo de Dato Abstracto) 5
Ejercicio 3: Pila (manejo estático por arreglo) 5
Ejercicio 4: Cola (manejo estático por arreglo) 5
Ejercicio 6: pila con punteros 6
Ejercicio 7: cola con punteros 7
Ejercicio 7 bis: lista con punteros 8
Ejercicio 8: pila con la biblioteca del IDE 9
Ejercicio 9: cola con la biblioteca del IDE 9
Ejercicio 10: lista con la biblioteca del IDE 10

Ejercicio 16: Ordenamiento Burbuja

```
template <typename T>
void bubbleSort(vector<T>& arr) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j + 1] < arr[j]) {
                T temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = temp;
            }
        }
    }
}
```

Desarrollo: Algoritmo que examina secuencialmente pares de datos vecinos. Si el dato contiguo es menor, los invierte creando un efecto de "flotación" para los valores extremos hasta alinear todo el grupo. Al utilizar un template <typename T>, la misma función se vuelve adaptable y logra operar independientemente si recibe un vector de caracteres o clases completas de estudiantes POO.

Ejercicio 17: Ordenamiento Merge Sort

```
template <typename T>
void mergeSort(vector<T>& arr, int izq, int der) {
    if (izq < der) {
        int medio = izq + (der - izq) / 2;
        mergeSort(arr, izq, medio);
        mergeSort(arr, medio + 1, der);
        mezclar(arr, izq, medio, der);
    }
}
```

Desarrollo: Se emplea la metodología de "divide y vencerás". El arreglo se corta repetitivamente en porciones la mitad de pequeñas hasta que no pueden fraccionarse más. La magia radica en la función mezclar, que recompone de regreso la línea evaluando ambas mitades y reinsertándolas linealmente y ya ordenadas en sub-arreglos temporales L y R.

Ejercicio 18: Ordenamiento Quick Sort

```
template <typename T>
void quickSort(vector<T>& arr, int bajo, int alto) {
    if (bajo < alto) {
        int pi = particionar(arr, bajo, alto);
        quickSort(arr, bajo, pi - 1);
        quickSort(arr, pi + 1, alto);
    }
}
```

Desarrollo: Utiliza la metodología matemática de pivoteo. Selecciona de forma arbitraria el dato extremo (pivote) e inserta a su izquierda todo lo que numéricamente sea menor, arrojando el resto mayor a la derecha mediante un intercambio directo de posiciones a través del método particionar.

Ejercicio 19: Ordenamiento Indirecto por Burbuja

```
template <typename T>
vector<int> bubbleSortIndirecto(const vector<T>& arr) {
    vector<int> indices(n);
    for (int i = 0; i < n; i++) indices[i] = i;
    // ... bucles burbuja evaluando arr[indices[j+1]] < arr[indices[j]]
    // y haciendo swap exclusivamente sobre los números de 'indices'
    return indices;
}
```

Desarrollo: Mover estructuras POO complejas exige mucha capacidad de procesamiento. Para evadir esto, se produce un "vector espejo" (lista numérica del 0 al n). Los algoritmos de evaluación leen los registros de los alumnos originales, pero los bloqueos y traslados de memoria los recibe este arreglo espejo secundario.

Ejercicio 20: Ordenamiento Indirecto por Merge Sort

```
template <typename T>
void mezclarIndirecto(const vector<T>& arr, vector<int>& indices, int izq, int medio, int der) {
    //... copia los indices a los sub-arreglos temporales L y R
    while (i < n1 && j < n2) {
        if (arr[L[i]] < arr[R[j]] || !(arr[R[j]] < arr[L[i]])) indices[k++] = L[i++];
        else indices[k++] = R[j++];
    }
}
```

Desarrollo: Cruza las ventajas recursivas de Merge Sort con el blindaje de memoria del ordenamiento indirecto. Reacomoda los parámetros direccionales en sub-listados referenciados (L y R), insertando de forma intercalada en el vector indices las posiciones actualizadas respetando el peso numérico medido en los objetos EstudiantePOO subyacentes.

Ejercicio 21: Ordenamiento indirecto por quick sort

```
void quicksort(T** arr, int bajo, int alto) {
    // ...
    if (*arr[j] < *pivote) {
        i++;
        swap(arr[i], arr[j]); // Se intercambian punteros
    }
}
```

Desarrollo: Se utiliza un arreglo de punteros (doble puntero en el caso de la clase genérica). El método selecciona el pivote dereferenciando la memoria (*arr[j]) para leer los valores reales, pero en el momento en que necesita ordenar, aplica el intercambio (swap) exclusivamente sobre las direcciones de memoria de los punteros.

Ejercicio 1: Punteros para el manejo indirecto con clases y estructuras

```
Auto vAuto[3];
Auto* pAuto[3];
for(int i=0; i<3; i++) pAuto[i] = &vAuto[i];
```

Desarrollo: Establece la base para el ordenamiento indirecto. Se crea un arreglo físico de objetos y, simultáneamente, un arreglo de punteros del mismo tamaño. A cada puntero se le asigna la dirección de memoria (&) de su objeto correspondiente, permitiendo acceder a ellos sin interactuar con la variable original.

Ejercicio 2: ADT (Abstract Data Type - Tipo de Dato Abstracto)

```
class ADT {
public:
    virtual void insertar(int valor) = 0;
    virtual void eliminar() = 0;
};
```

Desarrollo: Define un “molde” estricto utilizando funciones virtuales puras (= 0). No contiene lógica, solo establece las reglas de qué operaciones deben existir obligatoriamente (ej. insertar, eliminar, buscar) para que otras clases que hereden de ella (como Pilas o Colas) construyan el cómo.

Ejercicio 3: Pila (manejo estático por arreglo)

```
void push(int x) { if(tope < MAX-1) arreglo[++tope] = x; }
int pop() { return arreglo[tope--]; }
```

Desarrollo: Simula una pila de platos (LIFO: Último en entrar, primero en salir) limitando el espacio mediante un arreglo fijo. Se utiliza una variable entera llamada tope para registrar la posición del elemento más alto. Todas las inserciones y extracciones ocurren estrictamente en este índice tope.

Ejercicio 4: Cola (manejo estático por arreglo)

```
void enqueue(int x) { if(final < MAX-1) arreglo[++final] = x; }
int dequeue() { return arreglo[frente++]; }
```

Desarrollo: Simula una fila de supermercado (FIFO: Primero en entrar, primero en salir). Usa un arreglo con dos punteros lógicos (índices numéricos): final para registrar dónde se forman los nuevos datos, y frente para indicar qué posición es la próxima en ser extraída.

Ejercicio 6: pila con punteros

Desarrollo: una pila funciona con el criterio LIFO: el último en entrar es el primero en salir. Se implementó con nodos enlazados creados en memoria dinámica. Cada nodo (struct Nodo) guarda un dato y un puntero sig al nodo que tiene debajo. El puntero tope marca el nodo de arriba, y vale NULL cuando la pila está vacía.

push: crea un nodo que apunta al tope actual y lo convierte en el nuevo tope.

pop: guarda el tope en un puntero auxiliar, entrega el dato, baja el tope un nodo y libera el nodo con delete. Devuelve false si la pila estaba vacía.

El dato sale por referencia (int &x), que es lo que pide el enunciado con “considerando la referencia”. El constructor deja la pila vacía y el destructor la vacía para liberar toda la memoria.

```
class Pila {
    struct Nodo { int dato; Nodo *sig; };
    Nodo *tope;
public:
    Pila() : tope(NULL) {}
    ~Pila() { int x; while (pop(x)); }

    void push(int x) { tope = new Nodo{x, tope}; }

    bool pop(int &x) {
        if (!tope) return false;
        Nodo *aux = tope;
        x = aux->dato;
        tope = aux->sig;
        delete aux;
        return true;
    }
};
```

Ejercicio 7: cola con punteros

Desarrollo: una cola funciona con el criterio FIFO: el primero en entrar es el primero en salir. También se implementó con nodos enlazados, pero con dos punteros: frente (por donde sale el elemento) y final (por donde entra). Tener final evita recorrer toda la cola para agregar un elemento, y así encolar y desencolar no recorren la estructura.

encolar: crea el nodo, hace que el último apunte a él y lo marca como nuevo final. Si la cola estaba vacía, ese nodo es también el frente.

desencolar: entrega el dato del frente, mueve el frente al siguiente nodo y libera el anterior. Si la cola queda vacía, también se pone final = NULL. Si se olvida, final queda apuntando a memoria ya liberada (puntero colgante).

```
class Cola {
    struct Nodo { int dato; Nodo *sig; };
    Nodo *frente, *final;
public:
    Cola() : frente(NULL), final(NULL) {}
    ~Cola() { int x; while (desencolar(x)); }

    void encolar(int x) {
        Nodo *n = new Nodo{x, NULL};
        if (final) final->sig = n;
        else frente = n;
        final = n;
    }

    bool desencolar(int &x) {
        if (!frente) return false;
        Nodo *aux = frente;
        x = aux->dato;
        frente = aux->sig;
        if (!frente) final = NULL;
        delete aux;
        return true;
    }
};
```

Ejercicio 7 bis: lista con punteros

Desarrollo: una lista enlazada permite insertar, eliminar y buscar en cualquier punto, a diferencia de la pila y la cola. Se guarda un solo puntero, cab (la cabeza), y cada nodo apunta al siguiente.

insertarInicio: el nuevo nodo apunta a la cabeza actual y pasa a ser la cabeza.

insertarFinal: recorre hasta el último nodo y lo enlaza con el nuevo. Si la lista estaba vacía, el nuevo es la cabeza.

eliminar: usa un puntero doble (Nodo **p), que apunta al puntero que señala el nodo actual. Cuando se encuentra el valor, se hace que ese puntero salte al siguiente nodo y se libera el nodo. Así no hace falta guardar el nodo anterior ni tratar aparte el caso de borrar la cabeza.

buscar: recorre los nodos y devuelve true si encuentra el valor.

```
class Lista {
    struct Nodo { int dato; Nodo *sig; };
    Nodo *cab;
public:
    Lista() : cab(NULL) {}
    ~Lista() { while (cab) eliminar(cab->dato); }

    void insertarInicio(int x) { cab = new Nodo{x, cab}; }

    void insertarFinal(int x) {
        Nodo *n = new Nodo{x, NULL};
        if (!cab) { cab = n; return; }
        Nodo *p = cab;
        while (p->sig) p = p->sig;
        p->sig = n;
    }

    bool eliminar(int x) {
        Nodo **p = &cab;
        while (*p && (*p)->dato != x) p = &(*p)->sig;
        if (!*p) return false;
        Nodo *aux = *p;
        *p = aux->sig;
        delete aux;
        return true;
    }

    bool buscar(int x) const {
        for (Nodo *p = cab; p; p = p->sig)
            if (p->dato == x) return true;
        return false;
    }
};
```

Ejercicio 8: pila con la biblioteca del IDE

Desarrollo: se pide manejar una pila dinámica con las clases que ya trae el IDE. Se creó la clase Pila y, como atributo privado, un objeto stack<int> de la biblioteca <stack>, que maneja la memoria dinámica por dentro. Apilar se hace con push. Para sacar se lee el tope con top y se elimina con pop, revisando antes con empty que la pila no esté vacía. El valor sale por referencia, igual que en el ejercicio 6. No hay nodos ni punteros en el código.

```
#include <stack>
using namespace std;

class Pila {
    stack<int> datos;
public:
    void push(int x) { datos.push(x); }

    bool pop(int &x) {
        if (datos.empty()) return false;
        x = datos.top();
        datos.pop();
        return true;
    }
};
```

Ejercicio 9: cola con la biblioteca del IDE

Desarrollo: se pide lo mismo que en el 8, pero con una cola. Se creó la clase Cola con un atributo privado queue<int> de la biblioteca <queue>. Encolar se hace con push, que agrega al final. Desencolar lee el primero con front y lo elimina con pop, revisando antes que la cola no esté vacía. El valor sale por referencia. Tampoco hay punteros en el código.

```
#include <queue>
using namespace std;

class Cola {
    queue<int> datos;
public:
    void encolar(int x) { datos.push(x); }

    bool desencolar(int &x) {
        if (datos.empty()) return false;
        x = datos.front();
        datos.pop();
        return true;
    }
};
```

Ejercicio 10: lista con la biblioteca del IDE

Desarrollo: se pide una lista dinámica con las clases del IDE. Se creó la clase Lista con un atributo privado list<int> de la biblioteca <list>. Insertar al inicio y al final se resuelve con push_front y push_back. Para eliminar y buscar se usa find de <algorithm>, que devuelve un iterador: un objeto que funciona como un puntero a un elemento de la lista. Si find llega a end(), el valor no está. El iterador es lo único parecido a un puntero en este ejercicio, y la biblioteca maneja los nodos por dentro.

```
#include <list>
#include <algorithm>
using namespace std;

class Lista {
    list<int> datos;
public:
    void insertarInicio(int x) { datos.push_front(x); }

    void insertarFinal(int x) { datos.push_back(x); }

    bool eliminar(int x) {
        list<int>::iterator it = find(datos.begin(), datos.end(), x);
        if (it == datos.end()) return false;
        datos.erase(it);
        return true;
    }

    bool buscar(int x) const {
        return find(datos.begin(), datos.end(), x) != datos.end();
    }
};
```